TARS — Build Spec v0.1

Oct 17, 2026 · @Zara Rutherford

Goal and non-negotiables

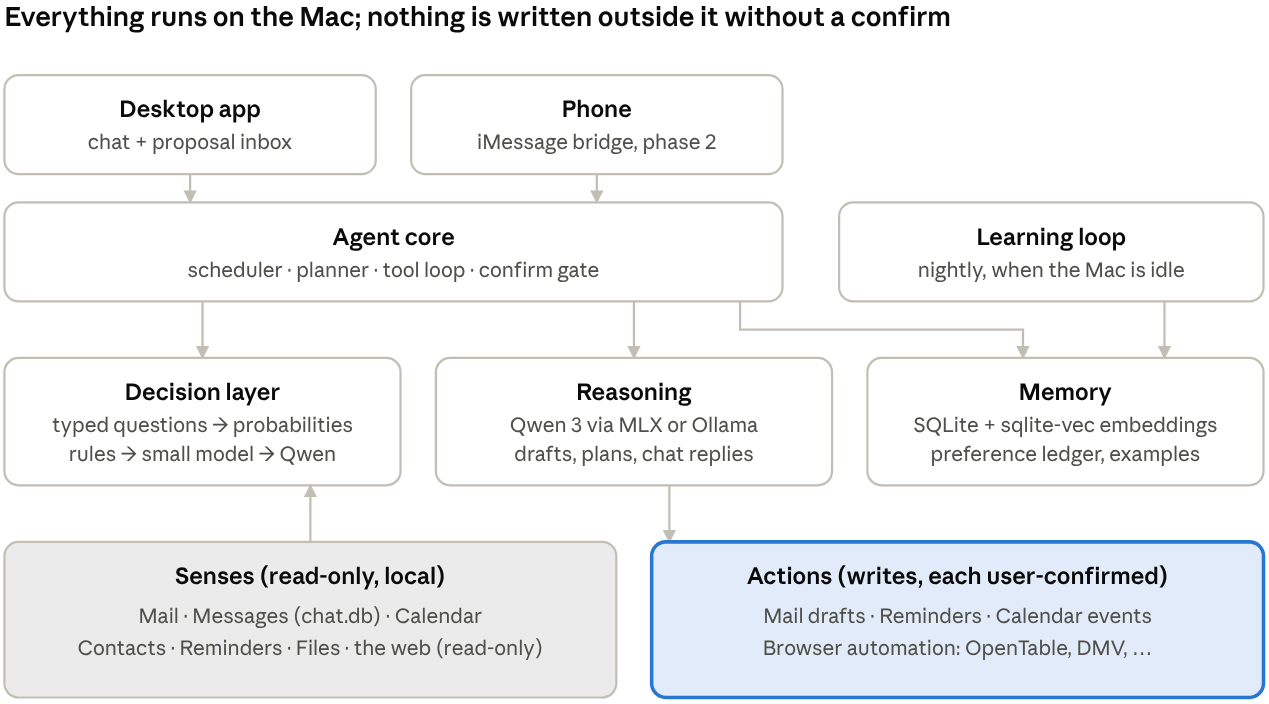
TARS is a personal assistant that runs entirely on the user's Mac, reads everything they have (mail, texts, calendar, contacts, reminders, files), and gets better at knowing what matters to them without any of it leaving the machine.

- **No third-party AI, ever.** Every model call (reasoning, classification, embeddings, speech) runs on local hardware. No API keys to any AI vendor in the codebase.
- **Read anything, write nothing without permission.** TARS can browse the whole internet, because reading leaks nothing. What it cannot do by default is send, post, submit, upload, or put the user's data into a request. Every write to the outside world waits for an explicit confirm from the user; the risk being managed is that TARS holds all of the user's information at once, so one unapproved write is a leak.
- **Drafts, not sends, by default.** TARS writes the follow-up to the professor; the user hits send. The user can later promote specific action types to auto-run.
- **Zero creator visibility.** TARS is a product. No telemetry, no sync, no account. The creator can't see any user's data because there is nowhere for it to go.
- **Swappable brains.** Every model sits behind an interface and is chosen in one config file, so TARS grows with the hardware under it.

TARS is a mold, not a finished product. For now it exists to be shaped: the user and TARS are figuring out together what a personal assistant should do, one feature and one correction at a time, until it fits the user exactly. Every rule, threshold, and behavior in this spec is a starting point meant to be tuned against real use, not a decision. Generalizing to other users comes after it fits one person well.

First milestone that counts: TARS notices an important email, pings the user, and has a draft reply waiting. Everything else builds on that loop.

Architecture



TARS architecture · 9 components, one write path

Senses pull everything into one local event store; the decision layer scores each event; reasoning only runs on what scores high; actions are the single door to the outside, and the learning loop retrains the decision layer from what the user does with TARS's proposals.

Stack. Python for the engine (best local-ML tooling: MLX, llama.cpp, sentence-transformers), a thin Swift helper for EventKit and notifications, Tauri or SwiftUI for the desktop app. One `tars.toml` config holds every model choice, threshold, and standing permission.

Event store. Every email, text, calendar entry, and reminder is normalized into an `events` table (source, sender, timestamp, body, thread id, embedding). Connectors are read-only watchers: Mail and Messages via their SQLite databases, Calendar/Contacts/Reminders via EventKit. TARS never writes back through a connector; writes go through Actions.

Proposal inbox. TARS never acts silently. Each thing it wants to do (flag, draft, reminder, booking) becomes a `proposal` row with a state: pending → accepted / edited / dismissed. The desktop app is mostly this inbox plus chat. Every state change is a training signal.

Classifier

Use Jev's *shape* (state + typed questions → calibrated probabilities), but run it locally as a three-tier cascade so the expensive model only sees what the cheap ones can't settle.

Interface. One function, `decide(state, questions) → {question: {answer, p}}`. Questions are typed: `noul` (yes/no), `choice` (one of N), `score` (ordered levels). Every backend implements this one interface, so swapping models never touches the rest of TARS. This is also the wire format of the hosted Jev API and its open clones (jeff/GLiFormer, JevOS), so a real local Jev drops in later with a config change.

The questions asked of every new event:

Question	Type	Used for
Does this need a reply from me?	noul	notify, draft
Urgency	score: none / today / this week	notify timing
Does this contain a commitment or plan?	noul	reminders
Who is this really from?	choice: VIP / known / cold / automated	weighting
Is this a reply I was waiting for?	noul	closes follow-up timers
Confidence	derived	escalation

Tier 0 — rules (µs). Deterministic and user-editable: VIP senders, calendar-linked threads, muted lists, "never flag newsletters". Rules come from the preference ledger, so the learning loop can write them too.

Tier 1 — small decision model (~20 ms). The workhorse. Start with a local embedding model (e.g. nomic-embed or bge-small, ~100M params) plus a per-question logistic head trained on the user's own labels. Later swap in jeff (GLiFormer, 400M) when the label count supports it. Runs on every event, CPU is fine. This is the model the learning loop retrains nightly.

Tier 2 — Qwen (~2 s). Only when tier 1's confidence lands in the uncertain band (say 0.35–0.65) or the event is from a VIP. Qwen gets the thread, the sender history, and the three most similar past examples with their labels, and answers the same typed questions in

constrained JSON. Its answers are written back as weak labels for tier 1.

Why this is the best approach for a laptop. 95% of events never wake the big model, so the laptop stays cool and classification is near-instant. And it scales cleanly: more compute means a bigger tier-1 model, a bigger tier-2 model, and a wider band that escalates to tier 2 — three numbers in the config.

Learning loop

The principle: the user should almost never have to teach TARS explicitly. Everything they already do is a label, if you watch it right.

1. Harvest implicit signals. Each proposal records what happened to it, but the richer signals are the ones nobody clicks:

- *Reply latency as urgency.* If the user replied to an email within 20 minutes, it was urgent, whatever TARS predicted. Replied after 3 days: not urgent. Never replied: probably didn't need one. This alone yields hundreds of labels a month for free.
- *Draft edit distance as a style oracle.* When the user sends a reply TARS drafted, diff the draft against what actually went out. Small edits → TARS's voice is right. Deleted paragraphs → too long. Changed greeting → wrong register for that person. Each diff becomes a per-recipient style note.
- *The false-negative catcher.* Any message the user replied to that TARS did *not* flag is the most valuable label there is. TARS logs these automatically and treats them as high-weight training examples.
- *Reminder outcomes.* A reminder dismissed inside 5 seconds was noise; one snoozed twice then completed was real but mistimed.

2. Shadow mode before every new behavior. No feature goes live blind. A new classifier or a new action type runs silently for two weeks, predicting what TARS *would* have done, then scores itself against what the user actually did. It graduates only when precision clears a threshold set in config. The user sees a one-line report: "Follow-up drafts would have been right 87% of the time. Turn on?" This is also how a new user's TARS earns trust on day one.

3. Cold start from history. On first run, TARS backfills from the last 6–12 months of mail and texts and applies the reply-latency rule retroactively. That produces a trained tier-1 model before the user has ever opened the app, so TARS is useful on the first morning, not the first month.

4. Per-person style fingerprints. From sent mail, TARS learns how the user writes *to each person*: length, formality, sign-off, whether they use the recipient's first name. Drafts are generated against the fingerprint for that recipient, not a global voice. New contact → nearest-neighbor fingerprint by relationship type (professor, friend, vendor).

5. A preference ledger the user can read. Everything TARS has concluded about the user lives in a plain-English file (`preferences.md`), one rule per line, each with its evidence: "Never flag Stanford mailing-list digests — dismissed 14 of 14." The user can edit or delete any line and TARS obeys. This replaces the black-box feeling with something closer to an assistant's notebook, and it makes the learning auditable.

6. Ask only where it's uncertain. Instead of asking for feedback on everything, TARS occasionally surfaces a borderline case: "I wasn't sure about this one — important or not?" Classic active learning; one such label is worth twenty easy ones. Capped at a few per day so it never nags.

7. Nightly consolidation. When the Mac is idle and plugged in, TARS replays the day: harvests the signals above, retrains the tier-1 heads, rebuilds the few-shot example bank, recalibrates confidence (predicted vs. actual over the trailing month), rewrites the preference ledger, and ages out examples older than ~6 months unless pinned. This job is where extra compute goes first: more compute, bigger model, more frequent runs.

8. Forgetting on purpose. A "this was a one-off" button on any proposal excludes that example from training. Rules in the ledger carry a last-confirmed date and decay if their evidence stops recurring. People change; TARS should too.

Portable to other users. Nothing above depends on the creator. Defaults ship in the config; the ledger, the examples, and the trained heads are files on the user's Mac. A user can even export their ledger to a new machine and keep their TARS.

Standing concerns

Some things don't arrive as an email; they sit in the back of the user's mind for months. TARS takes custody of them so the user can stop carrying them. The design goal is *fewer* interruptions, not more: one trusted "you're fine" a week is worth more than fifty reminders.

What a standing concern is. A durable goal with four parts: what done looks like, where the truth lives (a program sheet, a lease, a job tracker, a bank statement), what TARS watches for, and how often it reports. Stored as rows in a `concerns` table; the user adds

one in chat ("keep an eye on my graduation requirements") and TARS asks the few questions it needs to fill the four parts.

Concern	Truth lives in	TARS watches for	Reports
Graduate on time	program sheet, as a file the user gives TARS (no Axess login for now) + enrollment emails	courses taken vs. required, unit count, add/drop and graduation-application deadlines, a quarter where the remaining reqs don't fit	before each enrollment window, and when something slips
Decide: startup vs. SpaceX full-time (as a path to astronaut)	a decision file the user keeps (the criteria, what each path needs) + mail and calendar (SpaceX conversations, startup leads, return-offer timing)	dates that force the choice (return offer, application windows), new evidence for either side (a conversation, a lead, a milestone like finishing the degree), a thread going quiet	weekly one-liner on where the decision stands; a longer check-in only when a forcing date is within a month
Rent and bills paid	mail (invoices, autopay receipts), lease, calendar	due dates 3 days out with no receipt yet, an amount that changed, a late fee	only when something needs doing; otherwise "all paid" in the weekly note

The weekly note. Sunday evening, one message, one line per concern, green or amber: "Degree: on track, 9 units left, nothing due until Nov 15. Path: return-offer answer due Dec 1, two startup conversations open, nothing new this week. Money: rent paid, PG&E due Thursday, autopay on." If everything is green the note is three words long. Silence from TARS between notes means nothing is wrong, and TARS says that explicitly in onboarding so the user can believe it.

What TARS never does here. It never pays anything itself; money moves only when the user does it. It never contacts a recruiter or professor without the draft being approved. It never marks a requirement satisfied without a source — if the program sheet is

ambiguous it says "I'm not sure this counts; check with your advisor" rather than guessing.

Where it lands in the build. Concerns reuse the senses, the commitment extractor, and the draft engine; the new work is the `concerns` table, the watcher rules, and the weekly note. Build it right after M5 (Commitments), before the desktop app, since it's the feature that most changes how the user feels about TARS.

Compute profiles and scaling

TARS auto-detects the Mac's unified memory on first run and picks a profile; the user can override. Every profile is the same code with different model names and thresholds.

Profile	Memory	Tier 1 model	Tier 2 model	Escalation band	Nightly job
small	16 GB	bge-small + logistic heads	Qwen 3 8B, 4-bit	0.40–0.60	retrain heads only
medium	32–64 GB	jeff / GLiFormer 400M	Qwen 3 30B-A3B or 32B, 4-bit	0.35–0.65	heads + embedding fine-tune
large	96–192 GB	fine-tuned 1–2B decision model	Qwen 3 70B+, 8-bit	0.25–0.75	LoRA on tier 2 as well
future	more	whatever fits	whatever fits	wider	continuous

An M1 MacBook Pro with 16 GB is the *small* profile: an 8B model at 4-bit uses ~5 GB and leaves room for everything else. It will feel slow on tier-2 calls (a few seconds) but tier 1 handles almost everything, so day-to-day TARS feels instant.

What grows with compute, automatically: model size per tier, how often tier 2 is consulted, context window given to drafts, how much history the nightly job replays, and whether the big model itself gets fine-tuned. Nothing else changes.

Inference runtime. MLX on Apple Silicon is meaningfully faster than Ollama for the same model and supports LoRA fine-tuning natively; use it as the default and keep Ollama as a fallback backend behind the same interface.

Security and privacy

The goal is that "nothing leaves" is enforced by the system, not promised by the code.

- **Read-only internet by default, enforced outside the app.** All of TARS's browser and network traffic goes through a local proxy. Reads (GET, page loads) pass to any site. Writes (POST, PUT, form submissions, uploads, websockets) are blocked unless they carry a one-time approval token minted by the confirm gate for that specific action. The proxy also refuses any request whose URL or body contains text matching the user's recent events or contacts, so TARS can't leak by pasting an email into a search box. Everything blocked is logged. Ship it as a macOS network extension or a Little Snitch rule set so even a bug in TARS can't phone home.
- **No AI endpoints in the codebase.** A CI check greps for `api.openai.com`, `api.anthropic.com`, `api.typesafe.ai` and friends and fails the build if any appear. Trivial, and it's the kind of thing a privacy-conscious buyer will ask about.
- **Confirm gate on every outbound action.** Sends, bookings, form submissions, and payments wait for an explicit confirm in the app or by text. The user can promote individual action types to auto-run ("always add calendar events"); promotions are logged and revocable.
- **Audit log.** Every action, every outbound request, every model decision with its inputs: one append-only SQLite table. The app has a "what did TARS do today" view built on it.
- **Encryption at rest.** The event store, examples, and ledger live in an encrypted SQLite (SQLCipher) keyed from the macOS keychain. TARS's folder is useless if copied off the machine.
- **Permissions, once.** Full Disk Access (for Mail and Messages databases), Automation (for AppleScript), Calendar/Contacts/Reminders via EventKit prompts. TARS explains each prompt before it fires and runs with whatever subset is granted.
- **For the product:** no account, no license server, no telemetry, no update check that sends anything beyond a version number. Updates are signed; the app verifies the signature locally.

Build order

Each milestone is a straight-line dependency on the one before and ends with a demo you can show someone. Build these with Claude Code one at a time; don't start the desktop app until M3 works from the terminal.

1. **M0 — Engine runs.** MLX serving Qwen 3 8B behind a local HTTP endpoint; `decide()` interface stubbed with Qwen as the only backend. *Done when:* a script asks the five questions about a pasted email and gets typed JSON back.

2. **M1 — Senses.** Read-only connectors for Mail, Messages, Calendar, Contacts, Reminders into the `events` table, with a watcher that picks up new rows every 60 s. *Done when:* `tars events --since 1h` prints the last hour across all sources.
3. **M2 — Cold start + tier 1.** Backfill 6 months, derive reply-latency labels, train the embedding + logistic heads, wire the cascade. *Done when:* tier 1 beats a "flag everything from humans" baseline on held-out history.
4. **M3 — Notify.** Scheduler + macOS notifications for events that score high; proposal table with accept/dismiss from the notification. *Done when:* TARS pings you about a real important email before you see it. First daily-driver moment.
5. **M4 — Drafts.** Urgent-reply drafts and the no-reply-after-N-days follow-up, written to Mail as drafts with the per-person fingerprint. Draft-diff harvesting starts here. *Done when:* you send a TARS draft to a professor with fewer than three edits.
6. **M5 — Commitments.** Extract "lunch Thursday?" / "I'll send it tonight" into Reminders and Calendar, with reminder-outcome harvesting. *Done when:* TARS catches a plan you'd forgotten.
7. **M6 — Desktop app.** Chat + proposal inbox + preference ledger view + "what TARS did today". Tauri or SwiftUI. *Done when:* you stop using the terminal.
8. **M7 — Actions.** Playwright browser agent behind the egress proxy and confirm gate; first two targets: an OpenTable booking and the DMV appointment page. *Done when:* TARS books a dinner after you reply "yes".
9. **M8 — Phone.** iMessage bridge: TARS watches chat.db for messages from your number to a dedicated Apple ID and replies via AppleScript. *Done when:* you text "what's on tomorrow" from the car and get an answer.
10. **M9 — Productize.** Installer, permission walkthrough, profile auto-detect, signed updates, the no-AI-endpoint CI check. Then hand it to one other person.

Shadow mode (learning loop §2) gates M4, M5, and M7: each runs silently against your real traffic for two weeks before it's allowed to act.

Decisions

Settled on 2026-10-17; revisit any of these when they start to pinch.

Question	Decision
Which Mac runs it	The M1 MacBook Pro, 16 GB, small profile. A dedicated always-on Mac only if keeping the laptop awake gets annoying.
Mail source	Mail.app's local database. No IMAP credentials stored.
Desktop shell	SwiftUI menu-bar app over a Python daemon. Native gives the cleanest path to Full Disk Access, EventKit, Automation, and notifications; Tauri would route every one of those through a bridge. The daemon does all the thinking; the Swift app is permissions, notifications, and the chat/proposal window.
Learning loop defaults	All on. Nothing leaves the machine, so there's no reason to hold any of it back.
Business model	None for now. Make it a great tool for one person first.